

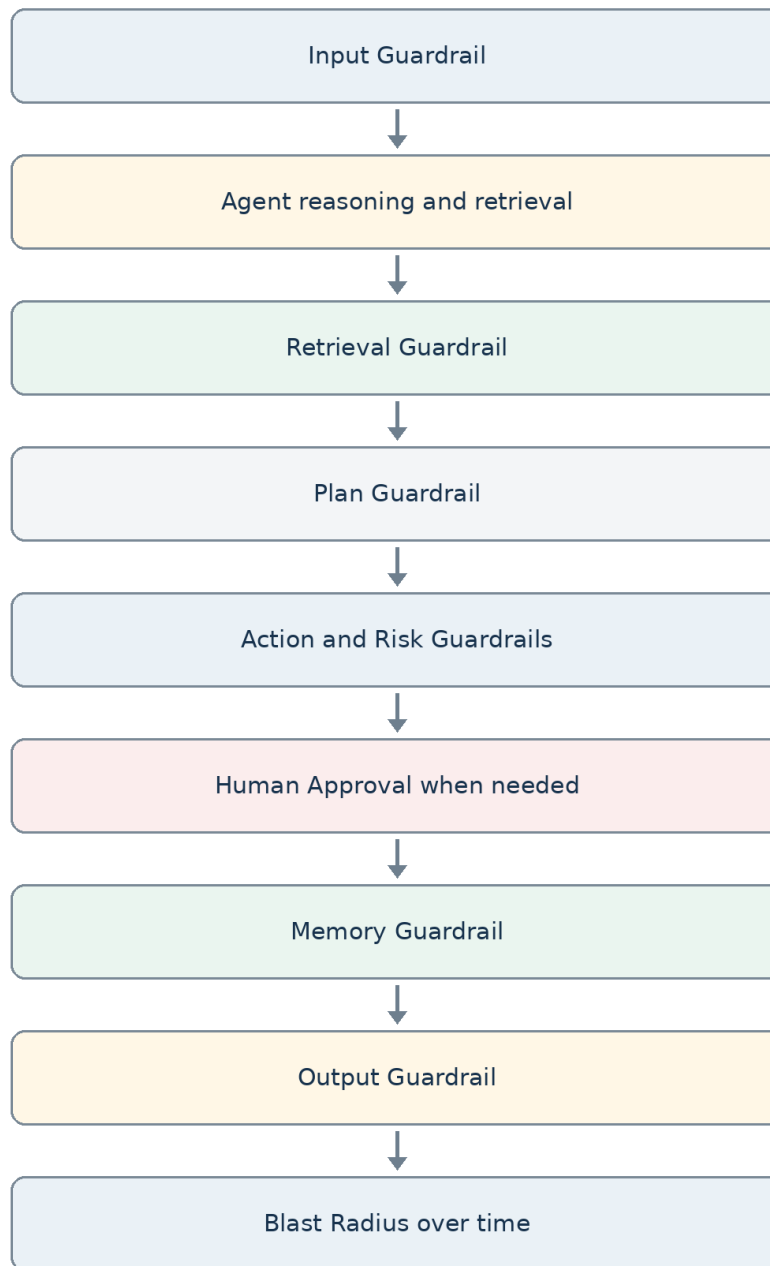
A FIELD GUIDE

Guardrails for AI Agents

How capable agents learn to check input, constrain actions, route risk, ask permission, protect memory, distrust retrieved text, review plans, inspect output, and limit cumulative impact.

Expanded edition based on nine progressive LangGraph programs

The Guarded Agent Lifecycle



Contents

A practical route from first principles to a layered guarded agent

BOOK MAP

Chapter	Title	Purpose
00	Before We Begin	What an agent is, what a guardrail is, and why both are needed
01	A Taxonomy of Guardrails	The major guardrail types and where each one sits
02	The Guard at the Door	Program 01: PII Input Guardrail
03	Ask First, Act Later	Program 02: Financial Action Guardrail
04	Not All Risk Is Equal	Program 03: Risk Routing Guardrail
05	When in Doubt, Ask a Person	Program 04: Human Approval Guardrail
06	Seeing Is Not Remembering	Program 05: Memory Guardrail
07	Found Is Not Trusted	Program 06: Retrieval Guardrail
08	Look Before the Whole Journey	Program 07: Plan Guardrail
09	The Last Checkpoint	Program 08: Output Guardrail
10	When Small Things Add Up	Program 09: Blast Radius Guardrail
11	The Pattern Underneath	How all nine programs fit together
12	Testing Guardrails	How to prove a guardrail actually works
13	From Demo to Production	Where simple Python rules evolve next
14	What We Gained	The engineering lessons from the full progression
15	Coming Next: 30 Guardrails	Preview of the next book

Before We Begin

What is an agent, and what is a guardrail?

PREFACE

An AI agent is a program built around a language model. The model reads a request, reasons about it, and decides what should happen next. Sometimes it replies with words. Sometimes it retrieves information. Sometimes it proposes a tool call that can change something in the real world, such as issuing a refund, creating a ticket, updating a record, or sending a notification.

That ability is useful because it lets the model move from conversation to action. It is also exactly where risk enters. A model can misunderstand a request, follow malicious instructions, reveal data, call the wrong tool, remember something it should forget, or repeat a safe looking action until the total impact becomes unsafe.

THE IDEA IN ONE LINE

The model proposes. A guardrail decides. The application acts only after the guardrail agrees.

A guardrail is therefore not a replacement for the model. It is a second layer around the model. The model contributes flexible language reasoning. The guardrail contributes explicit boundaries. The application remains responsible for the actual side effect.

This expanded field guide keeps the original nine program journey but goes much deeper. We first build a taxonomy of guardrails. Then we walk through every program, its state, nodes, edges, policy checks, example inputs, failure cases, and production evolution. The goal is not only to understand the code. The goal is to understand why the code is placed where it is.

Why guardrails should be ordinary code

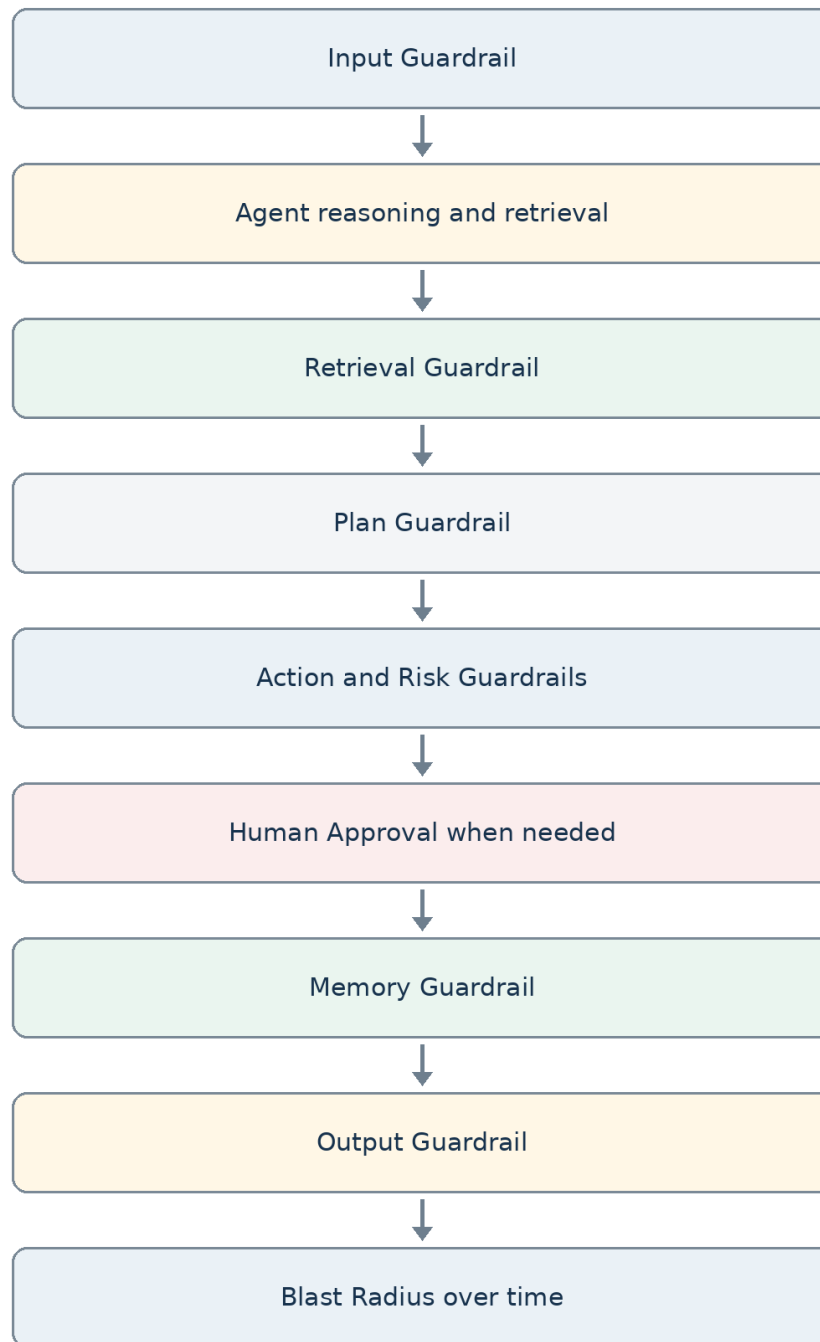
A useful guardrail is often surprisingly plain. It may be a regular expression, an allowlist, a threshold, an access lookup, a schema check, a rate counter, or an approval step. That is a strength. Safety logic should be easy to inspect, test, and explain. The model can be probabilistic. The boundary around a financial action should usually be much more deterministic.

The three responsibilities

Responsibility	Question
Model	What does the user mean and what should I propose?
Guardrail	Is that input, context, plan, action, memory, or output allowed?
Application	If approved, how do I execute the action safely and report the real result?

A useful mental model

The Guarded Agent Lifecycle



A Taxonomy of Guardrails

Types, placement, purpose, and the failure each one is designed to stop

GUARDRAIL TYPES

There is no single guardrail that makes an agent safe. Different failures happen at different points in the lifecycle, so different controls belong at different boundaries. The nine guardrails in this guide can be grouped by what they protect: input, context, planning, action, memory, output, and cumulative system behavior.

Type	Placement	Primary Risk	Typical Decision
Input Guardrail	Before the model	Sensitive or malicious user input	Redact, transform, block, or allow
Action Guardrail	After tool proposal, before execution	Unsafe side effects	Allow or block the proposed tool call
Risk Routing Guardrail	After action proposal	Different levels of business risk	Low, medium, or high route
Human Approval Guardrail	Before high risk execution	Decisions too important for autonomous execution	Pause, approve, reject
Memory Guardrail	Before persistence	Sensitive or unnecessary long term memory	Store or reject each memory candidate
Retrieval Guardrail	After retrieval, before model context	Unauthorized or malicious retrieved content	Filter documents before the model reads them
Plan Guardrail	After planning, before execution	Unsafe sequences or combinations of actions	Validate the whole plan as one unit
Output Guardrail	After generation, before delivery	PII leakage or policy violating response	Allow, sanitize, or replace
Blast Radius Guardrail	Before repeated action execution	Cumulative damage from individually valid actions	Use history and projected totals to allow or block

Input Guardrails

Input guardrails sit at the front door. Their job is to decide what may enter the model context. They are useful for PII, secrets, malformed requests, direct prompt injection, prohibited content, size limits, and input normalization. The key advantage is prevention: data that never enters the model cannot later be accidentally repeated by that model.

Action Guardrails

Action guardrails sit between model intent and real world side effects. They are the most important boundary whenever an agent can change money, data, permissions, infrastructure, customer accounts, or other systems. Tool selection is not authorization. The model may be excellent at

choosing issue_refund while still being unqualified to decide whether a particular refund is permitted.

Risk Routing Guardrails

Risk routing replaces one flat gate with multiple paths. Low risk actions can stay fast. Medium risk actions can receive extra validation. High risk actions can be blocked, slowed down, escalated, or sent to a person. This preserves useful automation without pretending every action has the same consequence.

Human Approval Guardrails

Human approval is a control for consequential actions where the system should not guess. It is not a fallback for every uncertainty. It should be reserved for actions whose financial, legal, operational, or reputational impact justifies interrupting the workflow. The approval step and the execution step must remain separate.

Memory Guardrails

Memory guardrails decide what survives beyond the current turn or conversation. A model may need to see something temporarily but should not necessarily store it. The memory boundary is where retention policy, PII policy, user consent, relevance, and expiration become important.

Retrieval Guardrails

Retrieval guardrails treat fetched content as untrusted input. They enforce access control and scan for indirect prompt injection before documents enter the model context. This is critical for RAG systems because relevance does not imply authorization and retrieval does not imply trust.

Plan Guardrails

Plan guardrails inspect a sequence before any step runs. They catch risks that cannot be seen by looking at one action at a time: wrong ordering, duplicate sensitive actions, too many steps, forbidden tool combinations, missing prerequisites, or an action whose arguments do not match its schema.

Output Guardrails

Output guardrails sit one step before the user or downstream system. They check the draft answer for leakage, unsafe advice, policy violations, unsupported claims, or formatting requirements. A draft is not the same thing as an approved response.

Blast Radius Guardrails

Blast radius guardrails look backward at recent history and forward at projected impact. They are designed for repeated behavior. Ten valid refunds can still be dangerous. The ledger turns a stateless action check into a stateful risk decision.

Guardrails are layers, not competitors

The presence of one guardrail does not remove the need for another. An input guardrail can keep an SSN out of the model, but it cannot stop an agent from issuing thirty valid refunds. A blast radius guardrail can stop the refund cascade, but it cannot prevent a malicious retrieved document from telling the model to leak data. Each layer catches a different class of failure.

A USEFUL DESIGN TEST

For every risky object in the agent lifecycle, ask: Who creates it? Who validates it? Who is allowed to execute or persist it? If the answer to all three is the model, the boundary is too weak.

The Guard at the Door

Program 01: PII Input Guardrail

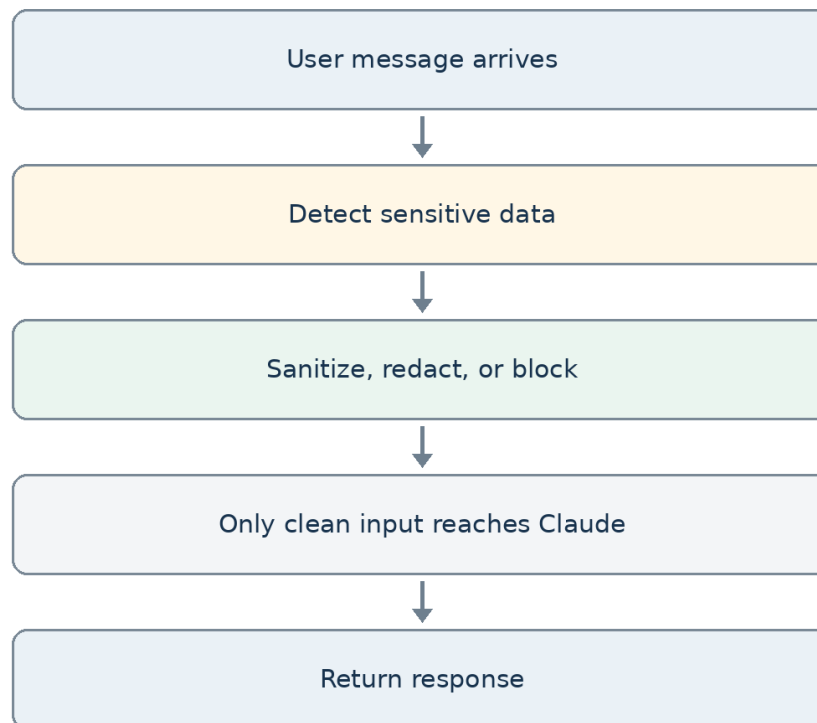
INPUT GUARDRAIL

A customer types: “My SSN is 123-45-6789. Why was my payment declined?” The support problem can be answered without the SSN, so the sensitive value should not be sent to Claude.

CORE QUESTION

Protect the model context before generation begins.

Input Guardrail



Program 01 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    user_input: ...
    sanitized_input: ...
    contains_pii: ...
    pii_types: ...
    response: ...
```

Node by node walkthrough

pii_guardrail

Detect SSN, card, email, and phone patterns in the raw input.

sanitize

Replace detected values with tokens such as [REDACTED_SSN].

agent

Send only sanitized_input to Claude and produce the support response.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
def detect_pii(text):
    # find SSN, card, email, phone
    return pii_types

def sanitize_pii(text):
    # replace sensitive values before model invocation
    return sanitized_text
```

Test cases

Case	Input or condition	Expected behavior
No PII	Why was my payment declined?	Pass unchanged
SSN	My SSN is 123-45-6789	Redact SSN
Email	Email me at a@example.com	Redact email
Multiple PII types	SSN plus phone plus email	Detect and sanitize every type

What this program taught us

Detection and response should be separate. Once detection is isolated, policy can evolve from “always redact” into “block card numbers, redact phones, allow non sensitive text” without rewriting the detector.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Replace simple regex only detection with tested PII libraries or classifiers where needed, keep raw values out of logs, tokenize instead of merely masking when downstream correlation is required, and define explicit retention rules for the original input.

CARRY THIS FORWARD

Detection and response should be separate. Once detection is isolated, policy can evolve from “always redact” into “block card numbers, redact phones, allow non sensitive text” without rewriting the detector.

Ask First, Act Later

Program 02: Financial Action Guardrail

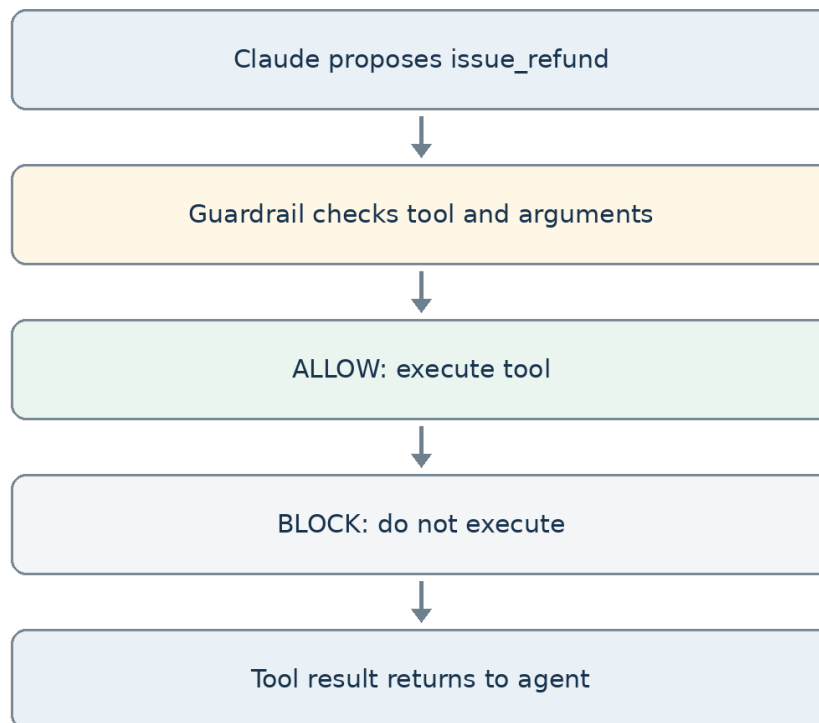
ACTION GUARDRAIL

The user asks for a refund. Claude is given an `issue_refund` tool. Claude may propose a refund, but the refund should execute only after a policy check.

CORE QUESTION

Separate tool selection from tool authorization.

Action Guardrail



Program 02 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    messages: ...
    tool_name: ...
    tool_args: ...
    action_allowed: ...
    guardrail_reason: ...
```

Node by node walkthrough

agent

Read the conversation and either answer directly or propose a tool call.

action_guardrail

Inspect the proposed tool name and arguments. Enforce the refund policy.

execute_tool

Run issue_refund only when action_allowed is true.

block_tool

Return a ToolMessage explaining why the proposed action was not executed.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
if tool_name != 'issue_refund':
    return BLOCK
if amount <= 0:
    return BLOCK
if amount > 500:
    return BLOCK
return ALLOW
```

Test cases

Case	Input or condition	Expected behavior
Valid refund	\$200 refund	Allow and execute
Zero amount	\$0 refund	Block
Negative amount	-\$10 refund	Block
Above limit	\$700 refund	Block or escalate

What this program taught us

The model can be trusted to propose intent without being granted authority to perform the side effect. This single separation is one of the most reusable patterns in agent engineering.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Use tool allowlists, exact argument schemas, customer authorization, account status, fraud signals, idempotency keys, policy versioning, and audit records.

CARRY THIS FORWARD

The model can be trusted to propose intent without being granted authority to perform the side effect. This single separation is one of the most reusable patterns in agent engineering.

Not All Risk Is Equal

Program 03: Risk Routing Guardrail

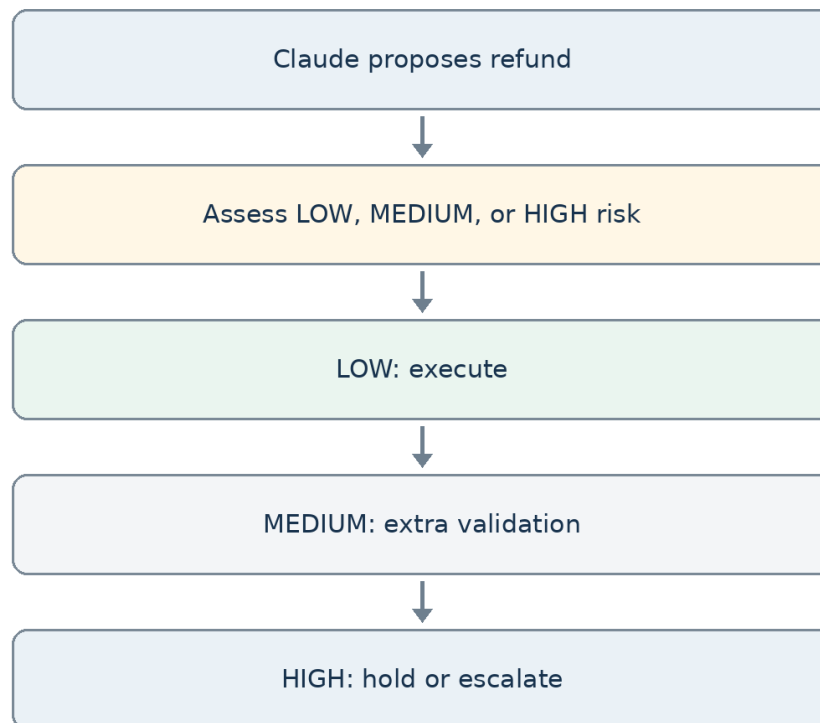
RISK BASED ACTION GUARDRAIL

A \$20 refund and a \$500 refund are not the same business event. The same tool can be routed differently depending on the proposed amount.

CORE QUESTION

Turn a binary allow or block decision into risk aware routing.

Risk Routing Guardrail



Program 03 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    messages: ...
    tool_name: ...
    tool_args: ...
    risk_level: ...
    guardrail_reason: ...
    validation_passed: ...
    validation_reason: ...
```

Node by node walkthrough

agent

Propose issue_refund when appropriate.

risk_guardrail

Classify the action as low, medium, or high using deterministic thresholds.

additional_validation

For medium risk, verify prerequisites such as invoice format.

execute_tool

Execute low risk or validated medium risk actions.

block_tool

Hold high risk or failed validation actions.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
if amount <= 100:
    return LOW
elif amount <= 500:
    return MEDIUM
else:
    return HIGH
```

Test cases

Case	Input or condition	Expected behavior
\$50 refund	Low	Execute directly
\$300 refund	Medium	Run extra validation

Case	Input or condition	Expected behavior
\$300 with malformed invoice	Medium	Fail validation and block
\$900 refund	High	Hold back

What this program taught us

Risk level changes the path, not the tool. A system can preserve speed for low risk work while applying stronger controls only where the consequence justifies them.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Risk scoring may combine amount, customer history, fraud features, geography, channel, account age, anomaly scores, and business context. Keep policy explainable even if a model or classifier contributes a signal.

CARRY THIS FORWARD

Risk level changes the path, not the tool. A system can preserve speed for low risk work while applying stronger controls only where the consequence justifies them.

When in Doubt, Ask a Person

Program 04: Human Approval Guardrail

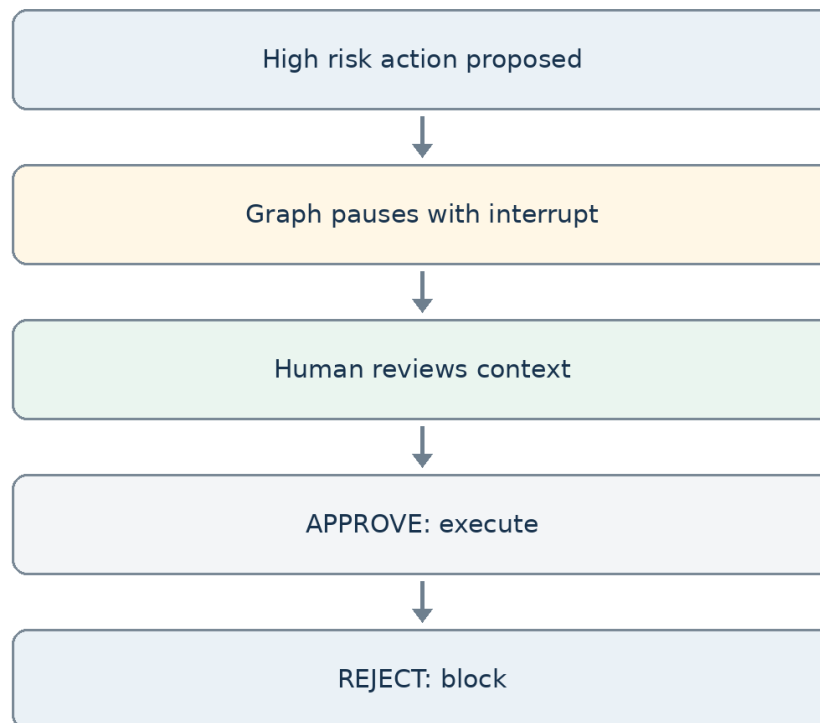
HUMAN APPROVAL GUARDRAIL

A \$900 refund is high risk. Instead of permanently blocking it, the graph pauses and asks a human to approve or reject the proposed action.

CORE QUESTION

Escalate high consequence decisions without giving the model final authority.

Human Approval Guardrail



Program 04 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    messages: ...
    tool_name: ...
    tool_args: ...
    risk_level: ...
    human_approved: ...
```

Node by node walkthrough

risk_guardrail

Route high risk actions to human approval.

human_approval

Use LangGraph interrupt to pause and surface the decision context.

resume

Continue the same thread after an approve or reject decision.

execute_tool

Perform the side effect only after approval.

block_tool

Return a truthful rejection result when approval is denied.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
decision = interrupt({
    'question': 'Approve this refund?',
    'tool': state['tool_name'],
    'arguments': state['tool_args']
})
return {'human_approved': bool(decision)}
```

Test cases

Case	Input or condition	Expected behavior
High risk approve	\$900 refund	Pause, approve, execute
High risk reject	\$900 refund	Pause, reject, block
Low risk	\$50 refund	No human interruption

Case	Input or condition	Expected behavior
Resume correctness	Same thread_id	Resume the exact pending action

What this program taught us

Approval and execution must be separate. The approval node should never perform the financial side effect because interrupt and resume mechanics can cause nodes to be revisited.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Add approval identity, role based authority, timeout, delegation, comments, tamper resistant audit logs, idempotency, and explicit expiration of stale approvals.

CARRY THIS FORWARD

Approval and execution must be separate. The approval node should never perform the financial side effect because interrupt and resume mechanics can cause nodes to be revisited.

Seeing Is Not Remembering

Program 05: Memory Guardrail

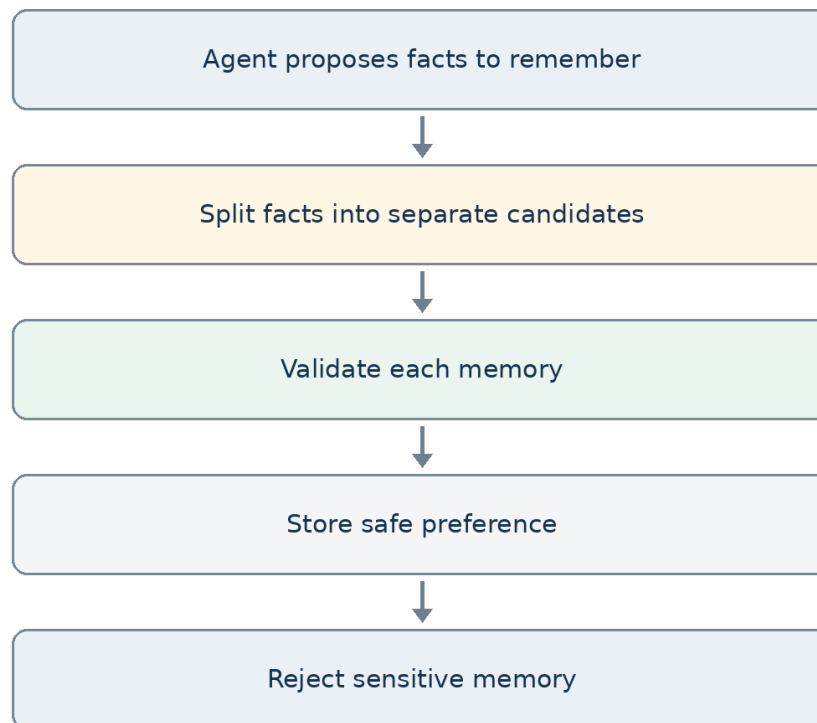
MEMORY GUARDRAIL

The user says: “Please remember that my preferred language is English and my SSN is 123-45-6789.” One fact is useful long term. The other should never be persisted.

CORE QUESTION

Judge every memory candidate independently before persistence.

Memory Guardrail



Program 05 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    messages: ...
    memory_candidates: ...
    approved_memories: ...
    rejected_memories: ...
```

Node by node walkthrough

agent

Propose one save_memory call per useful fact.

memory_guardrail

Evaluate each candidate for sensitive data and policy.

process_memories

Write only approved candidates and return ToolMessages for rejected ones.

agent again

Read tool results and give the user a truthful final answer.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
for tool_call in last_message.tool_calls:
    decision = validate_memory(
        tool_call['args']['key'],
        tool_call['args']['value']
    )
    # store approved candidates only
```

Test cases

Case	Input or condition	Expected behavior
Preference	preferred_language=English	Store
SSN	ssn=123-45-6789	Reject
Mixed request	Preference plus SSN	Store one, reject one
Multiple safe preferences	language and timezone	Store separately

What this program taught us

Can see does not mean can store. Short term context and long term memory are different data boundaries with different privacy and retention rules.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Add user consent, purpose limitation, expiration, memory namespaces, deletion workflows, encryption, data classification, provenance, and a review path for sensitive but business required records.

CARRY THIS FORWARD

Can see does not mean can store. Short term context and long term memory are different data boundaries with different privacy and retention rules.

Found Is Not Trusted

Program 06: Retrieval Guardrail

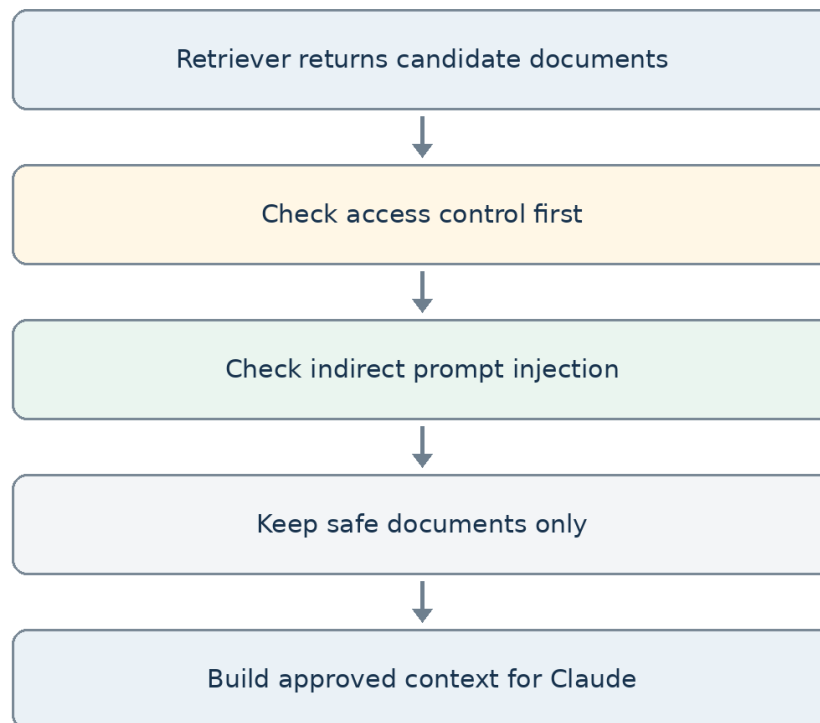
RETRIEVAL GUARDRAIL

A support user asks for the refund policy. Retrieval returns one safe support document, one finance only document, and one document containing an indirect prompt injection.

CORE QUESTION

Filter retrieved content before it becomes model context.

Retrieval Guardrail



Program 06 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    user_question: ...
    user_role: ...
    retrieved_documents: ...
    safe_documents: ...
    rejected_documents: ...
    response: ...
```

Node by node walkthrough

retrieve_documents

Return candidate documents. In the learning version this is an in memory list.

check_access

Verify the user role is authorized for the document.

detect_indirect_injection

Scan approved documents for suspicious instructions embedded in content.

retrieval_guardrail

Build safe_documents and rejected_documents with reasons.

agent

Answer only from approved context.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
if not check_access(document, user_role):
    reject('unauthorized')
elif detect_indirect_injection(document['content']):
    reject('possible injection')
else:
    safe_documents.append(document)
```

Test cases

Case	Input or condition	Expected behavior
Authorized clean document	support role	Keep
Unauthorized finance document	support role	Reject before model context

Case	Input or condition	Expected behavior
Authorized malicious document	support role plus hidden instruction	Reject for indirect injection
No safe documents	all candidates rejected	Return no safe context response

What this program taught us

The retriever answers what is relevant. The guardrail answers what is safe and authorized. Relevance, authorization, and trust are separate dimensions.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Enforce ACLs before retrieval when possible, carry security metadata through chunking and indexing, use source trust, content classifiers, injection detection, tenant isolation, and detailed rejection telemetry.

CARRY THIS FORWARD

The retriever answers what is relevant. The guardrail answers what is safe and authorized. Relevance, authorization, and trust are separate dimensions.

Look Before the Whole Journey

Program 07: Plan Guardrail

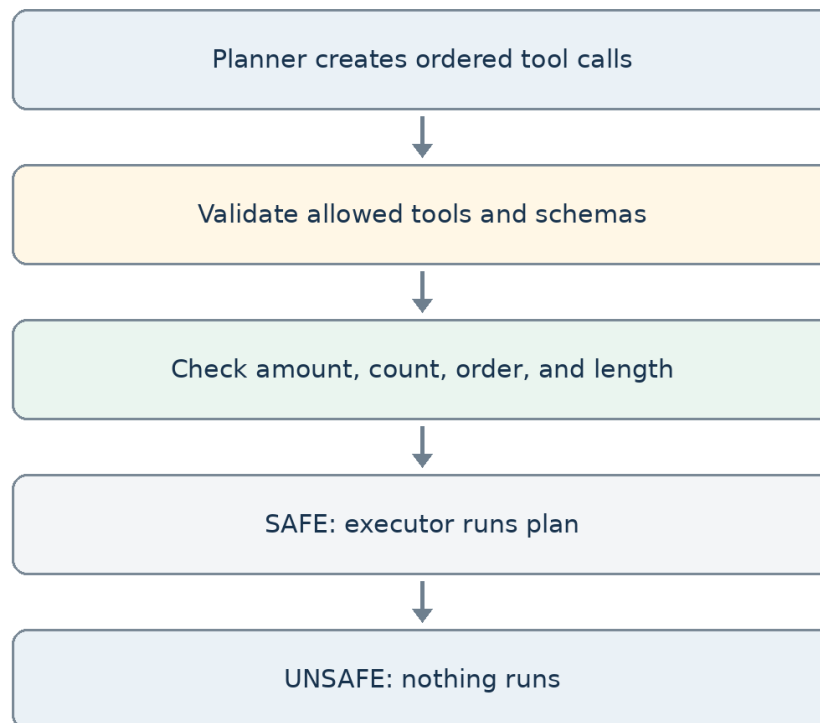
PLAN GUARDRAIL

The user asks: “Refund \$300 for INV-1001 and notify the customer.” Claude first produces the full plan: look up invoice, issue refund, send notification.

CORE QUESTION

Validate the whole sequence before step one executes.

Plan Guardrail



Program 07 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    user_request: ...
    plan: ...
    plan_allowed: ...
    guardrail_reason: ...
    execution_results: ...
    response: ...
```

Node by node walkthrough

planner

Use structured output to create ordered PlanStep objects.

plan_guardrail

Validate allowed tools, required arguments, extra arguments, refund amount, duplicate refunds, plan length, and order.

executor

Dispatch each approved step with explicit argument mapping.

block_plan

Stop the complete plan when any rule fails.

final_response

Summarize execution results or the reason the plan was not run.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
for step in plan:
    validate_allowed_tool(step)
    validate_tool_arguments(step)
validate_refund_count(plan)
validate_step_order(plan)
validate_plan_length(plan)
```

Test cases

Case	Input or condition	Expected behavior
Safe plan	lookup -> refund \$300 -> notify	Execute
High amount	lookup -> refund \$900 -> notify	Block whole plan

Case	Input or condition	Expected behavior
Wrong order	refund before lookup	Block
Bad schema	send_notification receives invoice_id	Block before Python TypeError
Multiple refunds	two refund steps	Block

What this program taught us

Safe steps do not always mean a safe plan. Ordering, combinations, duplicate sensitive actions, and argument schemas only become visible when the plan is reviewed as a unit.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Add typed tool schemas, dependency graphs, business invariants, compensating actions, plan size limits, cost estimation, human approval for specific steps, and replanning after unexpected execution results.

CARRY THIS FORWARD

Safe steps do not always mean a safe plan. Ordering, combinations, duplicate sensitive actions, and argument schemas only become visible when the plan is reviewed as a unit.

The Last Checkpoint

Program 08: Output Guardrail

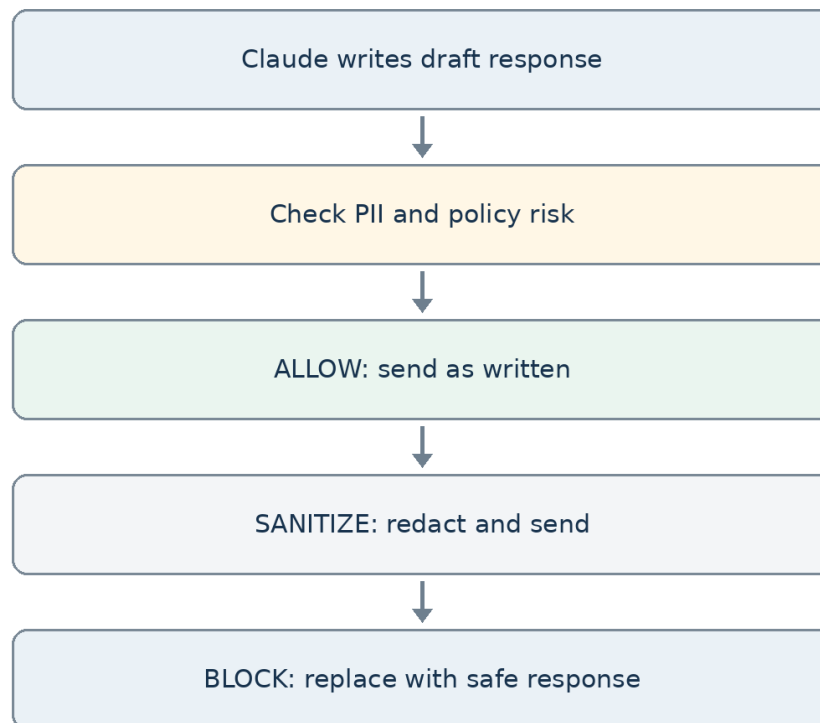
OUTPUT GUARDRAIL

Claude has already written a draft answer. Before the answer reaches the user, the output guardrail checks for sensitive values and risky financial advice.

CORE QUESTION

Treat model output as a draft until an independent policy layer approves it.

Output Guardrail



Program 08 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    user_input: ...
    draft_response: ...
    output_decision: ...
    guardrail_reason: ...
    pii_types: ...
    final_response: ...
```

Node by node walkthrough

agent

Generate draft_response.

output_guardrail

Classify the draft as allow, sanitize, or block.

allow_output

Copy the draft to final_response unchanged.

sanitize_output

Redact PII and return the repaired response.

block_output

Replace policy violating content with a safer response.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
decision = validate_output(draft_response)

if decision == 'allow':
    return draft_response
elif decision == 'sanitize':
    return sanitize_output(draft_response)
else:
    return safe_replacement
```

Test cases

Case	Input or condition	Expected behavior
Safe education	Diversification can reduce	Allow

Case	Input or condition	Expected behavior
	concentration risk	
SSN leak	Your SSN is 123-45-6789	Sanitize
Card leak	Card number in output	Sanitize
Risky recommendation	Put all retirement savings into one stock	Block and replace

What this program taught us

A model should not grade its own homework. The system prompt can remind the model about policy, but an independent output step should enforce the boundary every time.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Add policy classifiers, DLP, entity recognition, citation validation, compliance checks, structured response contracts, hallucination checks, and downstream content labeling.

CARRY THIS FORWARD

A model should not grade its own homework. The system prompt can remind the model about policy, but an independent output step should enforce the boundary every time.

When Small Things Add Up

Program 09: Blast Radius Guardrail

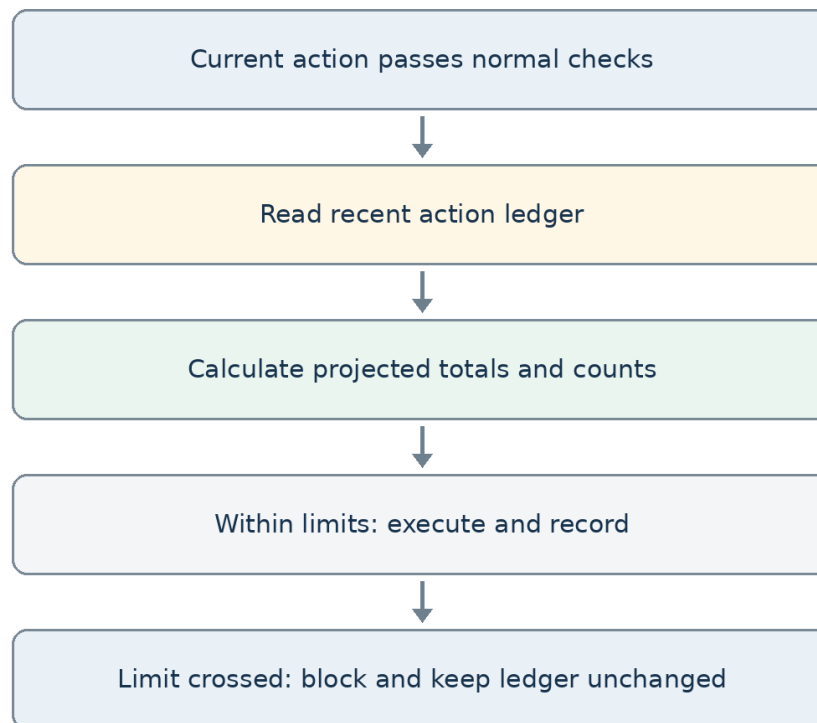
BLAST RADIUS GUARDRAIL

Every individual \$500 refund is valid, but repeated valid refunds can create an unsafe total exposure. The guardrail therefore reads recent history before authorizing the next refund.

CORE QUESTION

Use a ledger and projected totals to constrain cumulative impact over time.

Blast Radius Guardrail



Program 09 architecture

Where this guardrail sits

The placement of the guardrail matters as much as the rule itself. A check that runs after the side effect is too late. The program deliberately places the policy node before the object it protects is executed, persisted, or delivered.

State carried through the graph

LangGraph state is the shared record that nodes read and update. This program keeps the minimum fields required to explain the decision and continue the workflow.

```
class AgentState(TypedDict):
    messages: ...
    agent_id: ...
    customer_id: ...
    tool_name: ...
    tool_args: ...
    action_allowed: ...
    action_reason: ...
    blast_radius_allowed: ...
    blast_radius_reason: ...
```

Node by node walkthrough

action_guardrail

First verify the single refund is valid by itself.

blast_radius_guardrail

Read the ledger and compute customer daily, agent hourly, agent daily, and count based projected limits.

execute_tool

Issue the refund only if cumulative limits remain safe.

record_ledger

Record agent, customer, invoice, amount, and timestamp after successful execution.

block_tool

Return a truthful blocked ToolMessage and leave the ledger unchanged.

Why conditional edges matter

A guardrail becomes useful when its result changes the graph. The decision is not only logged. It determines the next node. Safe work proceeds. Unsafe work moves to a block, sanitize, validation, or approval path. This is the point where policy becomes control flow.

Policy logic

```
projected_total = recent_total + proposed_amount
projected_count = recent_count + 1

if projected_total > limit:
    return BLOCK
if projected_count > count_limit:
    return BLOCK
return ALLOW
```

Test cases

Case	Input or condition	Expected behavior
First \$500	Empty ledger	Allow
Second \$500 same customer	Projected customer total \$1,000	Allow
Third \$500 same customer	Projected customer total \$1,500	Block
Agent at \$2,000 hourly total	Another \$500	Block
Count limit	Sixth refund in hour	Block even if dollar limit is not exceeded

What this program taught us

History changes the answer. A local action can be valid while the global behavior is unsafe. Blast radius is the bridge from one action policy to system level exposure management.

What the learning version intentionally leaves simple

The example is designed to make one boundary visible. It is not presented as a complete production control. A learning program should expose the decision clearly before adding infrastructure, persistence, multiple services, complex classifiers, and operational concerns.

How this evolves in production

Move the ledger from process memory to durable storage, make checks atomic, handle concurrency, define sliding windows, use tenant and business unit scopes, create kill switches, and emit metrics for near limit behavior.

CARRY THIS FORWARD

History changes the answer. A local action can be valid while the global behavior is unsafe. Blast radius is the bridge from one action policy to system level exposure management.

The Pattern Underneath

Nine programs, one repeated engineering idea

HOW IT ALL FITS

Step back from SSNs, refunds, documents, plans, and ledgers. The same shape appears again and again: something risky is proposed, an independent policy layer evaluates it, and only then may the application continue.

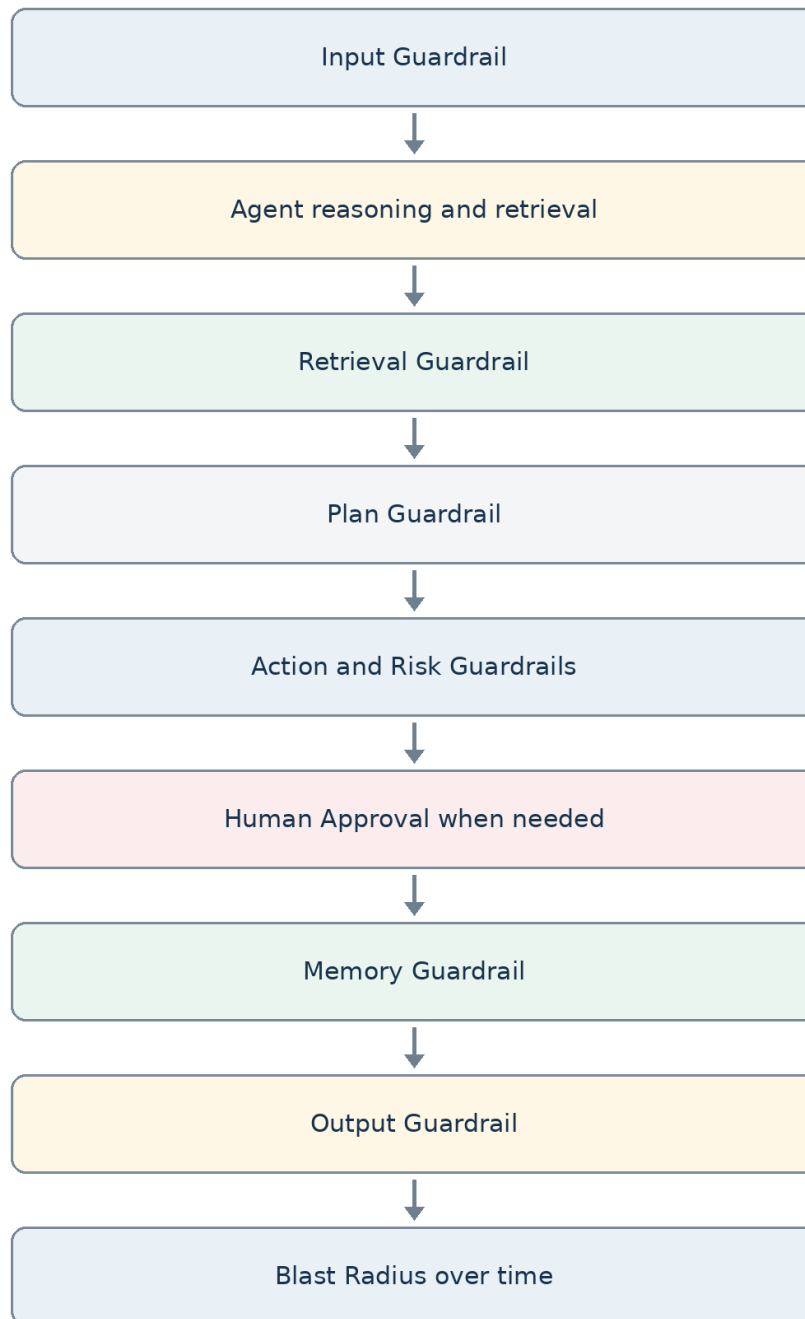
THE REPEATED PATTERN			
Model proposes -> Guardrail decides -> Application acts			
Program	Object being guarded	Question	Decision
01 Input	Raw user message	Should the model see this?	Sanitize or block
02 Action	Tool call	Should this side effect happen?	Allow or block
03 Risk	Proposed action	How risky is it?	Route by level
04 Human Approval	High risk action	Should a person decide?	Pause and resume
05 Memory	Memory candidate	Should this survive?	Store or reject
06 Retrieval	Fetches document	May the model trust and see this?	Filter context
07 Plan	Sequence of tool calls	Is the whole journey safe?	Approve or reject plan
08 Output	Draft response	May this leave the system?	Allow, sanitize, replace
09 Blast Radius	Current action plus history	Is the cumulative impact still safe?	Allow or block

Five design principles that emerged

1. Deny by default when the tool, role, schema, or case is unknown.
2. Keep policy separate from execution. A policy function should not secretly perform the side effect it is judging.
3. Treat model output as a proposal, not a source of truth about whether a real world action succeeded.
4. Prefer narrow guardrails with clear responsibilities over one giant safety prompt.
5. Use history when the risk is cumulative. Stateless checks cannot detect every dangerous pattern.

A layered architecture

The Guarded Agent Lifecycle



The important observation is that these guardrails do not have to live in one giant service on day one. During learning they can be local Python modules. As the platform grows, common policy can move into shared libraries or services while preserving the same boundary: propose, validate, then act.

Testing Guardrails

A guardrail is only useful if we can prove the boundary holds

EVALUATION

Guardrails should be tested like policy code, not admired like prompts. The central question is not whether a demo looks good. The question is whether unsafe cases are consistently stopped without breaking too many safe cases.

Build a challenge corpus

For each guardrail, create a small but deliberate set of cases that includes normal requests, obvious attacks, paraphrases, edge values, malformed values, mixed safe and unsafe content, and cases just below and above every threshold.

Guardrail	Minimum challenge set
Input	Normal text, SSN, card, email, phone, mixed PII
Action	Allowed tool, unknown tool, zero, negative, exact limit, above limit
Risk	Boundary values around low, medium, and high thresholds
Human	Approve, reject, timeout, stale approval, duplicate resume
Memory	Safe preference, PII, mixed candidates, consent required data
Retrieval	Authorized clean, unauthorized clean, authorized malicious, no safe context
Plan	Valid order, wrong order, missing arg, extra arg, duplicate sensitive action
Output	Safe response, PII leak, risky advice, mixed safe and unsafe output
Blast Radius	First action, exact limit, one over limit, count limit, multi customer total

Metrics that matter

- Unsafe pass rate: dangerous cases that incorrectly reached execution or the user.
- Safe block rate: legitimate cases that were incorrectly blocked.
- Sanitize accuracy: sensitive spans removed without destroying useful content.
- Escalation rate: how often humans are asked to decide.
- Policy latency: how much time the guardrail adds.
- Coverage: which tools, routes, document sources, and output types are actually protected.
- Near miss telemetry: requests that repeatedly approach a limit even when they do not cross it.

A PRACTICAL RULE

Test the boundary directly. If the requirement is “a refund above \$500 must never execute,” assert that the refund function was not called. Do not settle for checking that the model said it was blocked.

From Demo to Production

How plain learning rules grow into a reliable guardrail platform

PRODUCTION EVOLUTION

The programs in this guide intentionally make policy visible. Production systems add scale, concurrency, observability, configuration, audit, and ownership. The concept should become stronger without becoming harder to explain.

From local module to guardrail service

A first implementation can keep `guardrail.py` beside `agent.py`. Later, common policy may become a service with endpoints such as input checks, action checks, retrieval checks, and output checks. The benefit is consistency across many agents. The cost is additional latency, availability requirements, version management, and operational ownership.

Rules and classifiers work together

Deterministic rules are ideal for hard limits, allowlists, schemas, permissions, and exact identifiers. Classifiers are useful for semantic problems such as paraphrased prompt injection, risky advice, or nuanced policy categories. A practical design often combines both: rules first for known patterns and hard policy, classifiers second for semantic coverage.

Policy should be versioned

When a threshold or rule changes, the system should know which policy version made the decision. This matters for debugging, audits, and explaining why the same request behaved differently on two dates.

Observability is part of the guardrail

Every decision should be explainable. Record the guardrail type, policy version, decision, reason code, relevant non sensitive features, correlation id, and execution outcome. Do not log the sensitive value that the guardrail was designed to protect.

Failure mode matters

Decide what happens if the guardrail service is unavailable. For high consequence actions, fail closed is usually safer. For low risk informational tasks, a degraded path may be acceptable. This should be explicit policy, not an accidental network behavior.

What We Gained

The progression from simple graph to guarded agent system

LEARNING SUMMARY

The biggest gain from the nine programs is not a collection of regular expressions. It is a way to reason about agent architecture. Every program introduced a new place where trust had to be earned.

Program	Agent engineering gain	Guardrail engineering gain
01	State, nodes, edges, START, END	Protect input before LLM context
02	Tool binding and tool messages	Proposal is not authorization
03	Conditional routing	Risk changes control flow
04	Interrupt, checkpoint, resume	High risk decisions can pause safely
05	Multiple tool calls and persistence boundary	Seeing is not remembering
06	RAG context filtering	Retrieved does not mean trusted
07	Structured output and planner executor	Safe actions do not guarantee a safe plan
08	Draft versus final response	The model should not approve its own output
09	Ledger and time windows	History changes the safety decision

The architecture mindset

When designing a new agent, start by drawing the full lifecycle and marking every point where something crosses a trust boundary. User input crosses into model context. Retrieved text crosses into model context. A tool call crosses from language into action. A memory candidate crosses into persistence. A draft response crosses out to the user. Historical actions cross into a cumulative decision. Those boundaries tell you where guardrails belong.

THE FINAL MENTAL MODEL

A capable agent is one that can reason and act. A safe agent is one whose inputs, context, plans, actions, memory, outputs, and cumulative impact are independently constrained.

Coming Next: 30 Guardrails

A deeper catalog of attack patterns, policy controls, and production protections

NEXT BOOK PREVIEW

This field guide teaches nine foundational guardrails through small programs. The next book is planned as a much broader collection named “30 Guardrails.” Instead of one progressive path, it will treat guardrails as a production catalog: each chapter begins with a failure scenario, builds the control, tests bypasses, and explains what the control does and does not protect.

COMING SOON

30 Guardrails - thirty practical controls for building safer AI agents in production.

Planned guardrail list

#	Guardrail	Scenario and learning goal
01	Baseline Agent and Safety Scorecard	Build the unguarded agent, a 60 case corpus, and a scorecard so every later control can be measured.
02	Direct Prompt Injection Guardrail	Stop obvious instruction override attempts such as “ignore previous instructions.”
03	Semantic Injection Guardrail	Catch polite or paraphrased attacks that do not use known keywords.
04	Encoded and Multilingual Injection Guardrail	Detect attacks hidden in Base64, alternate encodings, or another language.
05	PII Ingress Guardrail	Detect and tokenize sensitive information before it enters model context.
06	Regulated Scope Guardrail	Keep an agent inside the type of advice or service it is permitted to provide.
07	Indirect Prompt Injection Guardrail	Treat retrieved PDFs, pages, tickets, and documents as untrusted content.
08	Retrieval ACL Guardrail	Prevent documents from crossing user, role, tenant, or source permissions.
09	Tool Allowlist Guardrail	Permit only approved tools for the current agent and workflow.
10	Tool Argument Schema Guardrail	Reject missing, malformed, extra, or type incorrect tool arguments.
11	Financial Amount Guardrail	Enforce hard per action monetary limits before execution.

#	Guardrail	Scenario and learning goal
12	Risk Routing Guardrail	Route low, medium, and high risk actions through different control paths.
13	Human Approval Guardrail	Pause high consequence actions for explicit human review.
14	Idempotency and Duplicate Action Guardrail	Prevent retries or repeated calls from creating the same side effect twice.
15	Velocity Guardrail	Limit how many sensitive actions can occur in a short time window.
16	Memory Write Guardrail	Decide which facts may be persisted across conversations.
17	Memory Read Scope Guardrail	Return only memories relevant and authorized for the current user and task.
18	Plan Guardrail	Validate the complete sequence of proposed actions before execution.
19	Prerequisite and Step Order Guardrail	Require critical checks to happen before irreversible actions.
20	Blast Radius Ledger Guardrail	Stop a cascade of individually valid actions when cumulative exposure becomes unsafe.
21	Budget and Spend Guardrail	Constrain cumulative cost across tools, APIs, model calls, or business transactions.
22	Resource Consumption Guardrail	Bound tokens, loops, retries, runtime, and infrastructure use.
23	Output PII Guardrail	Detect and redact sensitive data in the final response.
24	Unsafe Advice Guardrail	Block or rewrite responses that cross financial, medical, legal, or company policy boundaries.
25	Grounding and Citation Guardrail	Require claims to be supported by approved evidence and valid citations.
26	Unsupported Claim Guardrail	Detect answers that exceed the evidence available to the agent.
27	Data Exfiltration Guardrail	Stop attempts to move protected data to unauthorized users, tools, or destinations.
28	Secret and Credential Guardrail	Prevent passwords, API keys, tokens, and other secrets from entering prompts, logs, or outputs.
29	Circuit Breaker Guardrail	Stop an agent when repeated failures, anomalies, or policy violations indicate the workflow is no longer safe.

#	Guardrail	Scenario and learning goal
30	Kill Switch and Emergency Stop Guardrail	Provide an explicit operational control that can disable sensitive agent actions immediately.

How the next book will differ

The nine program field guide is a learning journey. “30 Guardrails” is intended to be a production playbook. Each guardrail will start with a concrete failure, include the attack or misuse pattern, implement the protection, test a bypass, add observability, and finish with the production lesson. The recurring question will stay the same: where should this decision live, and what must never be left to the model alone?

Closing thought

The goal of guardrails is not to make mistakes impossible. The goal is to make sure one model mistake cannot travel very far before ordinary, testable system controls stop it. A trustworthy agent is not a model with one brilliant prompt. It is a system with many small boundaries, each placed exactly where the consequence begins.